

The Digital Core Methodology

A configuration layer for collaborative work with language models — across research, software, and operations.

Wisdom Happy *Playful Sincerity Research Draft 2026-04-24 — provisional, pre-peer-review*

Keywords: Human–AI collaboration · Agent harnesses · LLM verification · Institutional memory · Configuration as epistemology · Tools for thought.

Author's Note

I am the sole author of this paper. I wrote it in collaboration with Claude (Anthropic), running inside Claude Code (Anthropic's CLI agent harness), shaped by the configuration layer this paper describes. Substantial portions of the prose were drafted by Claude under my direction; I edited, structured, and bear final responsibility for what is claimed. Where the paper makes a first-person observation about what the methodology feels like from the inside, that observation is mine — though Claude often articulated it before I did, and I kept the articulation when it captured what I'd seen.

The convention I am following — sole human author, AI collaborator named in acknowledgments and described throughout the body — matches the convention I use for other Playful Sincerity Research artifacts. It is not a hedge. It is a deliberate naming choice: when an AI system is doing something more than pattern-matching but less than independent authorship, the honest move is to name *the system* (Playful Sincerity Digital Core, henceforth PSDC) rather than the underlying model. PSDC is what produces work no individual component could produce alone. It deserves attribution; I deserve responsibility.

Abstract

Working alone with an AI collaborator inside a structured environment of my own design, I have produced verified results across multiple domains over the past several months — including a candidate algebraic framework that unifies nine classical mathematical domains, parts of a candidate physics framework, a graph-based memory architecture, a client-facing AI consultancy with custom software running inside each client's stack, and a contribution-

tracking system for distributed equity arrangements. The results are provisional and the paper itself is a working draft, not yet published. What made the breadth possible was a configuration layer I built and maintain: the **Playful Sincerity Digital Core (PSDC)** — persistent rules, on-demand skills, deterministic hooks, and a running chronicle plus surrounding document layer, sitting on top of Claude Code (Anthropic). The paper describes (i) PSDC’s four artifact types, (ii) the working patterns they enable, (iii) examples in use across domains, (iv) the intellectual lineage this work sits inside (Engelbart, Licklider, Clark, Hutchins, Alexander, second-order cybernetics, and the Zettelkasten tradition), (v) what this is and is not evidence for, and (vi) the broader paradigm — *digital core software* — that the paper sits inside.

1. Introduction

The claim is narrow. One researcher, working alone with a language model for three weeks inside a configuration layer of my own construction, produced provisional results in abstract algebra at a velocity that would plausibly have taken months for a traditional lone researcher. The Lean 4 formalization covers only first-order virtual numbers. The “division by zero” framing the work originally used turns out, on closer inspection, to be misleading, and I walked it back. Despite those caveats, I think the question this work raises is real:

When a researcher is embedded in a sufficiently structured AI environment, does “lone researcher” still mean what it meant in 2023?

I do not know. I am writing this paper to describe what I built, what I did, what I observed, and how it fits into a sixty-year intellectual lineage I think most of the current AI discourse has forgotten — so that other researchers, especially in academia, can decide for themselves whether to build something similar.

My honest view, after three weeks of intensive use across one mathematical case study and several adjacent research projects: a language model can be a real research partner for a researcher who has the discipline to build a verification layer around it. Without that layer, the model will generate plausible mathematics, plausible citations, and plausible proofs, and a researcher who trusts it gets a plausible paper full of errors. With the layer, the generation-plus-verification loop becomes something that moves faster than I think a single human or a naïve human-LLM pair can move.

Roadmap. Section 2 places this work in its intellectual neighborhood — agent harnesses, memory systems, research-pipeline automation, the cognitive-scaffolding lineage, and systems-design patterns. Section 3 describes PSDC itself: the four artifact types and how they compose. Section 4 describes the working patterns the configuration layer enables. Section 5

surveys the examples the methodology has produced across multiple domains. Section 6 discusses three broader themes that the prior-art survey surfaced. Section 7 is honest about what this is and is not evidence for. Section 8 names the broader paradigm — *digital core software* — that this paper sits inside. Section 9 closes with open questions.

Every non-trivial claim below is either computationally verified and reproducible from a public repository, internally consistent but not externally reviewed, or provisional hypothesis. I mark which is which.

2. Background and Related Work

PSDC sits at the intersection of five active research and engineering communities. None of them, in my reading, captures the full pattern. This section names where PSDC's components have prior art and where the integration is, as far as I have been able to verify, novel.

2.1 Agent harnesses and frameworks

An *agent harness* is the software layer that gives an LLM access to tools — file systems, shell, web, subagents — and orchestrates tool calls across a multi-turn task. Claude Code (Anthropic) is the harness PSDC runs on top of [1]. Comparable harnesses include Cursor [2] and Aider as IDE-integrated agents; AutoGen (Microsoft) [3] and LangGraph (LangChain) [4] as multi-agent orchestration frameworks; SWE-Agent (Princeton/Stanford, NeurIPS 2024) [5], OpenHands/OpenDevin [6], and Cognition's Devin [7] as task-specialized autonomous engineers; and CrewAI [8] as a role-based multi-agent system.

The distinction PSDC introduces is that it is *not* a harness. It is a configuration layer that operates on top of a fixed harness. The harness handles tool dispatch and execution; PSDC shapes how the model reasons within the harness — through always-loaded behavioral rules, persistent memory and chronicle files, and deterministic shell hooks that fire at session events. This is closer to the relationship between an operating system and an application than between two applications. To my knowledge, no published harness or framework treats the configuration layer as an independent, version-controllable, durable object in this way.

2.2 LLM memory and continuity systems

Persistent memory is an active research area. Letta (formerly MemGPT) [9] introduced an OS-style virtual-memory analogy with core and archival memory. Mem0 [10] extracts and indexes facts from interactions automatically. CoALA (Princeton) [11] proposes a four-quadrant memory

taxonomy (working, episodic, semantic, procedural) for language agents. Voyager (NVIDIA) [12] demonstrated lifelong skill accumulation in Minecraft. Stanford’s Generative Agents [13] used natural-language memory traces with reflection cycles for social simulation.

PSDC is adjacent to this neighborhood without being a memory system in the same sense. The chronicle and memory files are documents — markdown drafted by the AI as work happens, read by both AI and human, version-controlled in git. The operating principle, developed in §6.2, is that **storage is cheap and context is sacred**: rather than compress the record, write it readably and let any reader — human or future AI session — load only the parts that are relevant to the task at hand. The Zettelkasten tradition prescribed the same readable-by-the-later-reader property for paper notes [14, 15]; PSDC extends the property to a setting in which the dominant reader is also an AI.

2.3 AI-assisted research pipelines

Sakana AI’s *The AI Scientist* (2024) [16] and *AI Scientist v2* (2025) demonstrated end-to-end autonomous research pipelines: hypothesis, experiment, paper drafting. GPT Researcher [17] and Stanford’s Storm [18] automate literature synthesis and report generation.

These systems are designed for autonomy. PSDC is designed for the opposite: human-in-the-loop research where the AI is the velocity multiplier and verification is load-bearing. The model in PSDC does not produce a paper; it produces a *candidate* paragraph, formula, or argument that I check, push back on, or formalize. The verification layer is what keeps the speed from being hallucinated speed. None of the autonomous-research systems I have seen treats verification this way; they verify outputs at the end, not the inputs at every step.

2.4 Cognitive scaffolding and the extended mind

The deepest lineage PSDC sits inside is the one current AI discourse most often forgets. J.C.R. Licklider’s *Man-Computer Symbiosis* (1960) [19] explicitly described a system in which “men set the goals, formulate the hypotheses, determine the criteria, and perform the evaluations” while computers “do the routinizable work that must be done to prepare the way for insights and decisions in technical and scientific thinking.” Doug Engelbart’s *Augmenting Human Intellect: A Conceptual Framework* (1962) [20] introduced the H-LAM/T system — Human using Language, Artifacts, Methodology, and Training — and argued that intellectual capability is a function of the whole augmented system, not the human alone.

PSDC is, structurally, an instantiation of H-LAM/T. *Language* is shared natural-language convention plus the markdown formats. *Artifacts* are the chronicle, memory, and rule files. *Methodology* is the generate-verify-refine loop. *Training* is the configuration of the AI itself, plus the practitioner’s accumulated practice. I did not set out to implement Engelbart; I built the

system that worked, and only when researching prior art did I realize how exactly it matches the 1962 specification.

The philosophical scaffolding for why this works comes from Andy Clark and David Chalmers' *The Extended Mind* (1998) [21] and Edwin Hutchins' *Cognition in the Wild* (1995) [22]. Clark and Chalmers argued that cognitive processes are not bounded by skin and skull — when a notebook does the same job a memory does, the notebook is part of the cognitive system. Hutchins documented how distributed cognitive systems (a navigation team coordinating across instruments, charts, and crew) achieve performance no individual member could achieve alone. PSDC extends both arguments to a human-AI dyad: the chronicle, memory, rules, and skills are *part of the cognition*, not external aids to it.

The contemporary lineage on the human side is the tools-for-thought community — Bret Victor's *Inventing on Principle* and Dynamicland [23], Andy Matuschak's working notes [24], Roam Research, Obsidian, and the broader Zettelkasten revival [25]. PSDC's claim is that the same patterns extend to human-AI cognition, and that the AI's full participation requires the same kind of externalized, version-controlled, human-readable structure.

2.5 Systems design and pattern languages

PSDC's component vocabulary — rules, skills, agents, hooks, chronicle, memory — is a small pattern language in Christopher Alexander's sense [26]. Each pattern names a recurring problem in the human-AI workflow and a structural solution; the patterns compose. Donella Meadows' *Thinking in Systems* [27] gives the leverage-points framework: small structural changes (a behavioral rule, a Stop hook, a checkpoint) can shift system behavior more than large additions of content. Second-order cybernetics — particularly Heinz von Foerster's insistence that the observer is part of the system being observed [28] — is the operating philosophy: the chronicle is the system observing itself, and the breath rule is the system pausing to ask whether the observation is still right. Pattie Maes' *Concepts and Experiments in Computational Reflection* (1987) [29] anticipates the architectural pattern: the system carries an explicit model of itself that it reasons about and modifies.

Andrej Karpathy's recent work on “Software 3.0” [30] frames this period as one in which behavioral configuration of language models becomes the new programming. PSDC is one instance of what such configuration looks like when it is treated as serious infrastructure rather than a one-off prompt.

3. The Digital Core

PSDC has three structural layers. The **engine** is Claude — the language model that does the reasoning. The **harness** is Claude Code, which gives the model files, shell, web, subagents, and other tools. The **configuration** is PSDC: the rules, skills, hooks, chronicle, and surrounding documents I have authored on top of the harness to shape how the model works.

The engine and harness are Anthropic's. The configuration layer is mine. It consists of four kinds of artifact.

Rules. Always-loaded markdown instructions that shape how the model reasons. The current public set [31] includes rules for semantic logging (maintain a running narrative of what is happening and why), epistemic verification (do not claim a task complete until it has been observed working), model selection (route subagent tasks to cheaper models when possible), breath (pause periodically to check whether the current trajectory is still right), honesty (state what is true about where things are; never fabricate evidence), and safety scans on shell commands and fetched web content. From the inside, rules feel like durable biases. They are not features the model turns on; they are conditions it works under by default. Without them, the model drifts. With them, it mostly stays on-task and mostly catches its own mistakes. The rule layer is the least-discussed and, in my experience, the most consequential part of PSDC.

Skills. On-demand workflows invoked by slash command. Examples include `/debate` (spawn three adversarial sub-instances to stress-test a claim), `/play` (creative exploration with three voices — thread-follower, paradox-holder, pattern-spotter), `/plan-deep` (recursive decomposition of a large task), `/research-papers` and `/research-books` (multi-database literature search with raw-source preservation), and `/think-deep` (a meta-skill that composes research, play, structure, challenger, and synthesis into a single workflow). Skills keep complexity out of the main conversation until it is needed.

Hooks. Deterministic shell scripts that run when the model's turn ends. They check: has the chronicle been updated recently? has a breath-pause been taken? is the right model being routed for the current subagent task? The hooks emit reminders the model is then required to act on. This is self-regulation that does not depend on the model remembering to regulate itself, which matters because it frequently does not. Hooks are the *enforcement* layer for rules that require proactive action.

Chronicle and document layer. The chronicle is a narrative running record of work — kept per-project as dated files (`chronicle/YYYY-MM-DD.md` inside each project's directory) and at the ecosystem level for changes that span projects. Each entry records a decision, a discovery, an infrastructure change, a connection across projects. Around the chronicle sits the wider document layer described in §6.2: `MEMORY.md` plus per-topic memory files, per-project `ideas/` and `research/`, `knowledge/sources/` for primary inputs, `remote-entries/` for

capture from outside sessions. Together these close the gap that the model's statelessness leaves open. When a new conversation begins, the model's context is not empty; it is the prior work, written by the work itself, in files both AI and human can read. A small example of why this matters: an edit made in one conversation silently broke functionality being developed in a parallel conversation on the same project. Without the chronicle, the cause would have been false-attributed and time lost diagnosing the wrong thing; checking the chronicle pinpointed the change immediately, and the revert was a single step.

A sanitized public release of PSDC is available [31]; the full private system contains additional project-specific skills, rules, and memory not appropriate for public distribution.

4. Working Patterns

The configuration layer enables a small set of recurring patterns. None is novel in isolation; the value is in their composition under one operating discipline.

Generate-verify-refine. The model generates a candidate claim; the methodology runs a verification step (a test, a proof, a cross-source check, an adversarial debate); if verification fails, the generation refines and re-runs. The pattern is the standard scientific-computing loop. What is new is making it the *default* across an entire project — enforced by rules and hooks, including the meta-step of auditing the verification suite itself. The pattern matters most where the model's main risk shows up: confident fabrication. Plausible output is not evidence of correctness, and verification is what keeps speed from being hallucinated speed.

Multi-session decomposition. Work that exceeds a single conversation's capacity is split into parallel session briefs — self-contained markdown files that each drive a separate conversation, with explicit handoff points and synthesis at the end. This is how the more aggressive projects in §5 are structured, and it is how the methodology stays usable when no one conversation could carry the whole load.

Adversarial debate. For claims that resist computation — philosophical framings, naming decisions, design forks — three instances of the model with assigned positions argue the claim via the `/debate` skill. The convergence, or the remaining disagreement, is itself evidence about the claim's robustness. As a single instance, the model tends to converge too quickly toward agreement; as three instances with assigned positions, it produces more honest resistance.

Research-agent outputs preserved as primary data. Every research subagent's full output is saved as a dated markdown file and cross-referenced in any synthesis that uses it. Anyone reading the synthesis can trace a claim back to the specific agent that produced it. This hedges

against a failure mode the model is prone to: summarizing research in a way more coherent than the research itself, smoothing over real disagreements in the sources.

5. Examples in Use

The methodology has been applied across multiple domains. The point of this section is the breadth, not the depth of any single case — each example has a fuller writeup elsewhere; what they share is the same configuration layer.

From the research arm, the most developed example is **IVNA** (Indexed Virtual Number Algebra). With PSDC I formalized the core axioms in Lean 4, built a verification suite across six independent tool chains, and ran the prior-art search across more than thirty queries in seven search strategies. What the methodology helped me surface was the part that turned out to be genuinely novel: a single product rule, $\prod_x \cdot \prod_y = xy$, recurs across nine classical mathematical domains usually taught in isolation. The unification is the result. PSDC did not invent the math; it formalized it, surfaced its scope, and stress-tested specific claims through adversarial debate. One concrete instance of the methodology earning its keep: a scheduled audit of the verification suite caught a passthrough function that had been silently ‘passing’ twenty derivative tests by computing nothing — exactly the failure mode the model is prone to and the methodology exists to catch. The full IVNA writeup, including verification details and external review, lives in its own repository [32]. A **graph-based memory architecture**, a **candidate physics framework** (developed via the most aggressive multi-session decomposition we have run), and a **long-running formal-language project** are three more research-arm examples.

From the operations and consulting arm, **Happy Human Agents** is a client-facing AI consultancy where the agentic system lives inside the running software each client uses; the **Customer Relationship Builder (CRB)** is one concrete example of that new pattern, and §8 returns to it. A **contribution-tracking system** for distributed equity arrangements is a second operations-side example. Other domains — creative, infrastructure, personal — have also been worked through PSDC and have produced outputs not yet ready to describe in detail.

Depth on any one of these belongs to its own writeup. What the present paper claims is that the same configuration layer has produced reproducible, verified, and useful outputs across all of them.

6. Discussion: Three Broader Themes

The prior-art survey in §2 surfaced three themes the methodology embodies but had not, before this paper, named explicitly. I name them here because I think they are the parts most worth carrying forward, regardless of whether PSDC itself catches on.

6.1 The 1960s vision, finally instantiable

I did not know about Licklider or Engelbart when I built PSDC. I built the system that worked under daily stress. The connection surfaced when the AI ran the prior-art research for this paper — agent searches across academic databases turned up *Augmenting Human Intellect* (1962) and *Man-Computer Symbiosis* (1960), and on reading them I realized that Engelbart's H-LAM/T architecture maps almost element-for-element onto PSDC. The chronicle is the artifact layer, the rules and skills are the methodology layer, the model itself is the language layer, and my accumulated practice is the training layer. Licklider's symbiosis claim — humans set goals, formulate hypotheses, evaluate; machines do the routinizable work — is an exact description of what the verification loop looks like in practice.

The honest gloss: PSDC found this synthesis, not me. That matters for what it implies. If the architecture is discoverable from the structure of the problem rather than inherited from theory, then I should expect many more independent rediscoveries converging on the same pattern: stateless engine + harness + persistent configuration + human-readable substrate. The 1960s vision *required* sufficiently capable underlying components to instantiate. Until language models capable of meaningful tool use existed, the configuration layer had nothing to configure. Now it does. PSDC is one early instantiation; any team that takes the problem seriously will reinvent something in this neighborhood.

6.2 Storage is cheap, context is sacred

The most important shift PSDC makes in the memory neighborhood is to stop calling it memory. The chronicle, the `MEMORY.md` index, the per-topic memory files — these are documents. Markdown files. The AI drafts them as work happens; the operator reads and corrects them; both parties read them later. The operating principle is *storage is cheap, context is sacred*: prose committed to disk costs almost nothing, while loading the wrong thing into the model's context window costs both quality and attention. The work is to write the right files in the right places, not to compress them.

Reading the same substrate as the writer is the design constraint. A future AI session has no prior context; a six-weeks-later human collaborator has no recent memory. Both need to be able to open a file and understand it. That property — readable by anyone who later needs to

use it — is what the Zettelkasten tradition has prescribed for sixty years for paper notes. PSDC carries the constraint into a setting where the readers include AI sessions, not just humans, and where the writer is often the AI rather than the operator.

‘Memory’ inside PSDC is not one store. It is a small family of stores, each tuned to a different temporality and audience: the **per-day chronicle** carries the narrative of what happened and why; the `MEMORY.md` **index plus per-topic memory files** carries facts that survive across sessions; per-project `ideas/` captures overflow during focused work; `research/` holds raw source materials and agent outputs; `knowledge/sources/` preserves human speech and other primary inputs; `remote-entries/` stages capture from outside live sessions. Folder structures, file conventions, and links between files let the system load only what is relevant for a given task — most files stay on disk and out of context until they are needed. This is not a new idea; the file system has worked this way for decades. What PSDC contributes is the discipline to apply it consistently for human-AI collaboration, with documented filing rules per store. None of this is the final memory system; it is a practical approach calibrated to how the current Claude Code harness works best. The principle that outlasts the specifics is simple: organize information so that humans and agents can traverse it together. The linkages between nodes of the digital core — how files reference each other, how indexes route to detail, how on-demand loading is decided — are still an open optimization frontier.

6.3 Verification-narrative integration

The strongest claim I want to make about PSDC is one the prior-art survey supported by failing to find a precedent. Existing systems handle verification as an output: they run tests, they produce a pass/fail. They handle narrative as a separate output: they generate a paper, a report, a summary. *They do not integrate the two.* The chronicle in PSDC is not a log of what was done. It is the record of *why* a verification step was taken, what alternatives were considered, what surprised the operator, what was checked because the system flagged it, and what was deferred. The narrative *is* the proof that the verification was real and not theatrical.

The meta-audit of IVNA’s `ivna_derivative()` passthrough function (§5) is the canonical example. Without the chronicle, the bug would have shipped. The chronicle did not catch the bug; the scheduled audit caught the bug. But the audit was a scheduled step *because* the chronicle from prior weeks had documented the verification suite’s growth and flagged the meta-verification step as worth running. The narrative produced the audit; the audit caught the bug; the chronicle now contains the bug-and-fix story for future reference. That is the integration I mean.

I do not yet know if “verification-narrative integration” is the right name for this. I do know that it is what makes the methodology worth describing.

7. What This Is Evidence For, and What It Is Not

What this provisionally supports. A single operator, working with a sufficiently capable language model in a structured environment that emphasizes verification and persistent documentation, can move faster through real work than the same operator without that environment — across domains that include mathematical research, physics, formal language, client-facing software, and operational systems. The verification layer is what keeps the speed from being hallucinated speed. The chronicle and document layer is what makes the work survivable across sessions and across operators.

What it does not support. It does not show that AI systems will replace research teams or product teams. It does not show that any of the case examples in §5 are correct or important; for IVNA, peer review has not happened. It does not show the methodology generalizes to every domain; the examples are biased toward formalizable, computationally tractable, and document-heavy work.

Who this is for. Earlier drafts scoped the audience to academic researchers in heavily computational fields. The actual audience is wider. Anyone considering building or operating custom software with an embedded agentic editor — researchers, founders, consultants, operators, individuals running their lives on a custom stack — has a stake in whether the methodology lands. The strengths it offers are the same in every case: breadth, speed, patience, audit trail, continuity across sessions and operators. The failure modes are also the same: confident fabrication, premature convergence, tests that pass without testing. All can be mitigated if the operator treats verification, narrative, and the document layer as load-bearing rather than decorative. §8 names the broader paradigm this paper sits inside.

8. Digital Core Software

The category this paper sits inside is bigger than the methodology, and worth naming directly. People have been arriving at it from different angles for a little while now under different terms; the name *digital core software* is one consolidation.

What it is. Custom software with an agentic system inside it that the user owns and can keep editing — not as a separate sysadmin chore after deployment, but during use, as part of using it. The build/use distinction collapses. The user is not a tenant of someone else's SaaS; they are an operator editing the thing they are using while they use it, with an AI agent doing most of the editing under their direction. The data lives in their own system.

A digital core, more precisely. A *digital core* is the configuration and document layer that sits on top of the agentic system inside the software: rules that shape how the agent reasons, skills that compose into workflows, hooks that enforce discipline, and a chronicle and document layer that carries institutional memory across sessions and across operators. PSDC is one instance. Many other configuration layers exist; PSDC appears to be one of the more advanced multidisciplinary ones, but the category does not depend on PSDC.

What it changes. The business model shifts: instead of selling subscriptions to a shared SaaS instance, the practitioner sells *consulting* to set up the digital core inside the client's own software, after which the client owns it, modifies it, and may receive updates from a community of people who write modules for that digital core. **Happy Human Agents** is one early instance of this model; the **Customer Relationship Builder (CRB)** is one concrete software piece operating inside a client's stack under that arrangement.

Why it matters now. Until language models capable of meaningful tool use existed, the configuration layer had nothing to configure and the agentic-editor-inside-the-software pattern had no engine. Now both exist. The pattern is being independently arrived at by multiple practitioners. The integration of the pieces — agentic editor + user ownership + document-layer continuity + consulting-not-SaaS economics — is what is new; the pieces themselves are not. PSDC is one early instantiation. What this paper contributes is the methodology layer that makes a digital core competent and trustworthy in daily use.

9. Open Questions

- Is there a theorem IVNA proves that nonstandard analysis does not? The single most important open question about the mathematics, and the one most likely to determine whether the framework deserves mathematical-contribution status.
- Does the methodology scale beyond mathematics to empirical sciences, where verification is noisier and slower? My intuition is yes, with adaptations — the verification layer has to absorb statistical, not just symbolic, evidence — but I have not tested this.
- What is the right form of external validation for work produced this way? Traditional peer review, collaborative replication by another AI-plus-human pair, formal theorem provers — each gives different guarantees. The right answer is probably *all three, in sequence*, but I do not yet know how to formalize that.
- Can the methodological patterns in PSDC be extracted into a form other researchers can adopt without re-doing the months of environment-building the system represents? The

sanitized public release [31] is an attempt at this; whether it lands depends on whether the patterns are as portable as I think they are.

- How does the configuration layer evolve as model capability evolves? PSDC's specific rules and skills are calibrated to the failure modes of present-day language models. As the failures change, the configuration will need to change. The pattern — rules + skills + hooks + chronicle + document layer — should outlast the specific contents.

Reproduction. The IVNA repository is public. The PSDC Meta System is public [31]. Every verification check cited in §5 can be re-run from the IVNA repository.

A closing note. This paper is itself produced using the methodology it describes. A `/think-deep` session scoped it. A `/research` round produced the prior-art survey in §2. Agent outputs are saved alongside this draft. The chronicle for 2026-04-19 logged the original drafting decisions; the chronicle for 2026-04-24 logs this revision. The recursion is intentional: the work documents the tool that produced the work. What I hope a careful reader takes away is not that I am a capable researcher, though I may or may not be, but that the environment I built around the language model made the work possible in a way the research community has not fully registered yet.

Acknowledgments

This paper, like the IVNA work it describes, was developed in collaboration with the **Playful Sincerity Digital Core** — a research methodology system I have built on top of Claude Code (Anthropic). Substantial drafting assistance came from Claude (Anthropic) operating within this methodology; I am the author and bear final responsibility for the claims, but PSDC and the underlying model were essential to the work's velocity and to the prose's clarity. Kiel Howe (Stanford) provided the first external review of IVNA on 2026-04-13; his question about the subscripts is reshaping the framework. Errors are mine. Any successes are the methodology working the way it is supposed to.

References

[1] Anthropic. *Claude Code overview*. <https://code.claude.com/docs/en/overview> ; Anthropic, *Building agents with the Claude Agent SDK*. <https://www.anthropic.com/engineering/building-agents-with-the-claude-agent-sdk>

- [2] Cursor (Anysphere). *Cursor — The AI Code Editor*. <https://cursor.com/>
- [3] Wu, Q., Bansal, G., et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. Microsoft Research, 2024. <https://github.com/microsoft/autogen>
- [4] LangChain. *LangGraph — durable, stateful agent orchestration*. <https://github.com/langchain-ai/langgraph>
- [5] Yang, J., Jimenez, C. E., et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024. <https://arxiv.org/abs/2405.15793>
- [6] All-Hands AI. *OpenHands (formerly OpenDevin) — Code Less, Make More*. <https://github.com/All-Hands-AI/OpenHands>
- [7] Cognition AI. *Introducing Devin, the first AI software engineer*. 2024. <https://cognition.ai/blog/introducing-devin>
- [8] crewAI Inc. *CrewAI — Framework for orchestrating role-playing, autonomous AI agents*. <https://github.com/crewAIInc/crewAI>
- [9] Packer, C., Wooders, S., et al. *MemGPT: Towards LLMs as Operating Systems*. 2023. <https://arxiv.org/abs/2310.08560> ; Letta. <https://github.com/letta-ai/letta>
- [10] Mem0 AI. *Mem0 — The Memory Layer for Personalized AI*. <https://github.com/mem0ai/mem0> ; Mem0 paper: <https://arxiv.org/abs/2504.19413>
- [11] Sumers, T. R., Yao, S., Narasimhan, K., Griffiths, T. L. *Cognitive Architectures for Language Agents*. Princeton, 2023. <https://arxiv.org/abs/2309.02427>
- [12] Wang, G., Xie, Y., et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. NVIDIA, 2023. <https://voyager.minedojo.org/>
- [13] Park, J. S., O'Brien, J. C., et al. *Generative Agents: Interactive Simulacra of Human Behavior*. Stanford, ACM UIST 2023. <https://arxiv.org/abs/2304.03442>
- [14] Luhmann, N. *Communicating with Slip Boxes*. (Trans. by M. Kuehn.) Discussion of the Zettelkasten method behind Luhmann's ~90,000-note research output.
- [15] Ahrens, S. *How to Take Smart Notes*. 2017. The contemporary canonical treatment of the Zettelkasten tradition.
- [16] Lu, C., Lu, C., Lange, R. T., et al. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. Sakana AI, 2024. <https://sakana.ai/ai-scientist/> ; *AI Scientist v2*, 2025.
- [17] GPT Researcher. *Autonomous research agent for comprehensive online research*. <https://github.com/assafelovic/gpt-researcher>
- [18] Shao, Y., Jiang, Y., et al. *Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models (STORM)*. Stanford, NAACL 2024. <https://github.com/stanford-oval/storm>

- [19] Licklider, J. C. R. *Man-Computer Symbiosis*. IRE Transactions on Human Factors in Electronics, 1960. <https://groups.csail.mit.edu/medg/people/psz/Licklider.html>
- [20] Engelbart, D. C. *Augmenting Human Intellect: A Conceptual Framework*. SRI International, 1962. <https://www.doungengelbart.org/pubs/augment-3906.html>
- [21] Clark, A., Chalmers, D. *The Extended Mind*. Analysis 58(1), 1998. <https://consc.net/papers/extended.html>
- [22] Hutchins, E. *Cognition in the Wild*. MIT Press, 1995.
- [23] Victor, B. *Inventing on Principle*. CUSEC 2012. <https://vimeo.com/906418692> ;
Dynamicland. <https://dynamicland.org>
- [24] Matuschak, A. *Working notes*. <https://notes.andymatuschak.org>
- [25] Roam Research, Obsidian, and the broader tools-for-thought community. The Zettelkasten revival in personal knowledge management 2018–present.
- [26] Alexander, C., Ishikawa, S., Silverstein, M. *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press, 1977.
- [27] Meadows, D. *Thinking in Systems: A Primer*. Chelsea Green, 2008. *Leverage Points: Places to Intervene in a System*. The Sustainability Institute, 1999.
- [28] von Foerster, H. *Cybernetics of Cybernetics, or, the Control of Control and the Communication of Communication*. Future Systems Inc., 1995. The canonical statement of second-order cybernetics.
- [29] Maes, P. *Concepts and Experiments in Computational Reflection*. OOPSLA 1987. <https://dl.acm.org/doi/10.1145/38765.38821>
- [30] Karpathy, A. *Software 2.0* (Medium, 2017) and subsequent talks on Software 3.0 framing language-model configuration as the new programming paradigm.
- [31] *Playful Sincerity Digital Core — Meta System*. Sanitized public release. <https://github.com/Playful-Sincerity/PSDCMS-Playful-Sincerity-Digital-Core-Meta-System>
- [32] *IVNA — Indexed Virtual Number Algebra*. Public repository for the IVNA framework, including verification suites across six tool chains, the Lean 4 formalization, and the standalone IVNA writeup. <https://github.com/Playful-Sincerity/IVNA-Indexed-Virtual-Number-Algebra>

Word count (body): ~5,200